

## 7-2 Project Two: Summary

During Project One, I used a thorough and organized unit testing method for the Appointment, Contact, and Task services. My strategy was all about checking constructor rules, making sure setter limits were followed, and confirming that service-level actions like adding, updating, and deleting worked appropriately. For the appointment service, I looked at tricky cases like null dates, past dates, and very long descriptions using `assertThrows`, making sure that any bad inputs were rejected. In the contact service, I checked the lengths of fields and null rules for all attributes, and I tested updates for both specific fields and the whole contact. The task service has a similar approach, with tests aimed at checking constructor rules, update logic through field selectors, and generating unique IDs.

My testing method was tightly aligned with the software requirements. Each service had strict rules about input length and null values, just like the rubric stated. For instance, in `AppointmentTest.java`, I created `testLongDescription()` to reject descriptions that were longer than 50 characters. In `ContactServiceTest.java`, I checked the `updateContact()` method with selection values and null checks. In `TaskServiceTest.java`, I used `assertThrows` to catch any invalid updates and deletions. These tests made sure that the application would work reliably in both expected and unexpected situations.

The quality of my JUnit tests was impressive, with almost complete coverage of constructors, setters, and service methods. I used a combination of `assertThrows`, `assertEquals`, `assertTrue`, and `assertNotNull` to check both the expected behavior and edge cases. My tests were modular, clear, and followed the best practices of test-driven development. For instance, I reused helper methods

like `futureDate()` in `AppointmentServiceTest.java` to make the test setup easier and avoid repeating myself. I also checked the object state after updates using assertions like `assertTrue(contact.getName().equals("Justin Smith"))`, which helped confirm that the updates were correct without needing to hardcode multiple fields.

To make sure that everything is technically solid, I put the validation logic in the constructors and setters. This way, my tests could concentrate on how things behave instead of getting caught up in the details of how they work. I kept things efficient by steering clear of unnecessary logic and by using dynamic ID generation along with helper methods. In the Contact service, I used `Vector`, and in the Task service, I applied stream filtering, which showed a scalable way to handle service logic. I made sure to test everything thoroughly to ensure it was robust.

## 7-2 Project Two: Reflection

In this project, I used a variety of important software testing techniques. I applied boundary testing to check the maximum and minimum lengths for IDs, names, and descriptions. For exception testing, I utilized `assertThrows` to make sure that any invalid inputs were correctly rejected. I also conducted state verification through assertions that confirmed the object's state after updates. These techniques were selected because they are relevant to input validation and enforcing business logic in mobile applications.

There were some techniques that I didn't use in this project, like mocking and integration testing. Mocking is more suited for testing external dependencies or APIs, which weren't included in this assignment. Integration testing, although crucial in systems that involve multiple services or

databases, was beyond the scope of this unit-level project. I also skipped parameterizing testing, but I ensured thorough coverage through manual test case design.

Each technique has its practical uses depending on the context of the project. Boundary and exception testing are vital for validating user input in mobile apps, where reliability and user experience are key. State verification is important to ensure that business logic functions as intended, which is crucial for maintaining data integrity. Mocking and integration testing become more significant in larger systems with external dependencies, where isolated unit tests might not provide a complete view.

Throughout this project, I kept a cautious and detail-focused mindset. I looked out for edge cases and invalid inputs, showing a defensive testing strategy. I made sure to separate the testing logic from the actual implementation, which helped reduce bias and maintain objectivity. For instance, I tested `updateContact()` with invalid selection values to mimic user mistakes, and I checked `generateUniqueID()` to confirm it met uniqueness and length requirements. These tests showed that I understood the complexity and connections within the code I was evaluating.

I also worked hard to minimize bias when reviewing the code. As a developer, it's tempting to think your own code functions perfectly, but I pushed back on that idea by creating tests aimed at possible failure points. I considered how a developer might miss certain constraints and wrote tests to identify those issues. This approach allowed me to stay objective and enhance the overall quality of my code.

Lastly, I understand how crucial it is to be disciplined in my dedication to quality as a software engineer. Skipping steps in testing can lead to technical debt, which affects maintainability and reliability. I intend to steer clear of technical debt by consistently writing modular, validated, and

well-documented code. I will also keep up the practice of writing thorough unit tests before deployment, making sure my code fulfills both functional and non-functional requirements.

### References

I did not use any sources for this project